

Section 2 — Building Your First Model

What is a Pydantic Model?

A Pydantic model is a structured blueprint that defines what data an application expects.

A model specifies:

- Field names
- Field data types
- Required fields
- Optional fields
- Default values
- Validation behavior

For example, a student model may define:

- name as a string
- age as an integer
- marks as a float

When data is passed to the model, Pydantic checks whether the data follows the defined structure.

The basic flow is:

Incoming Data



Pydantic Model



Validation



Valid Model Object

If validation fails:

Incoming Data



Pydantic Model



Validation Failed



ValidationError

BaseModel

BaseModel is the main base class used to create Pydantic models.

A custom Pydantic model is normally created by inheriting from BaseModel.

Conceptually:

BaseModel



Custom Model



Fields and Types



Validation

After inheriting from BaseModel, Pydantic can:

- Read field definitions
- Validate incoming values
- Convert compatible values

- Apply default values
 - Generate validation errors
 - Create structured model objects
-

Fields

A field represents one piece of data inside a model.

For example, a user model may contain:

User

```
|  
├— name  
├— age  
├— email  
└— active
```

Each field has a name and an expected type.

Example concept:

name → str

age → int

salary → float

active → bool

Pydantic uses these type annotations during validation.

Required Fields

A field without a default value is required.

Conceptually:

name: str

means that a valid value for name must be provided.

If required data is missing, validation fails.

Flow:

Required Field

↓

Was a value provided?

↓

Yes → Validate the value

No → ValidationError

Required fields are useful for information that must always exist.

Examples:

- Username
 - Product name
 - Order ID
 - Email address
 - Customer name
-

Default Values

A field can have a predefined default value.

If the user does not provide that field, Pydantic uses the default automatically.

Concept:

Field value provided?

↓

Yes No
↓ ↓

Use input Use default

Default values are useful for fields such as:

- Country
- Account status
- Active status
- Experience
- Quantity
- User role

For example, an account may be active by default.

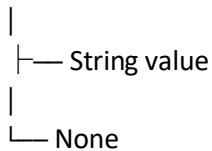
Optional Fields

Sometimes information may not be available.

An optional or nullable field can accept a normal value or None.

Concept:

phone



This is useful for:

- Phone numbers
- Profile pictures
- Portfolio links
- Secondary addresses
- Middle names
- Optional descriptions

A commonly used pattern also gives the field a default value of None, allowing it to be omitted from input.

Nullable and Omittable Fields

These two concepts should not be confused.

A **nullable field** can contain None.

An **omittable field** can be left out when creating the model because it has a default value.

Conceptually:

Nullable

Value can be:

actual value

OR

None

Omittable

Field may be:

provided

OR

missing because a default exists

A field can be both nullable and omittable.

Type Coercion

Pydantic can convert compatible values into the expected type.

For example:

Input	Expected	Result
"25"	int	25
"45000.50"	float	45000.5
1	bool	True

This behavior is called **type coercion**.

The flow is:

Incoming Value



Correct Type?



Yes No



Accept Can Convert?



Yes No



Convert Error

Type coercion is useful because external data often arrives as strings.

For example:

- Form data
- Environment variables
- URL parameters
- JSON from external systems

Lenient Validation

By default, Pydantic often behaves like a lenient checkpoint officer.

If a value has the wrong type but can be safely interpreted as the required type, Pydantic may convert it.

Example concept:

Expected: Integer

Received: "25"

Result:

"25" → 25 → Accepted

This makes applications easier to work with when data comes from external sources.

Strict Validation

Sometimes automatic conversion is not desirable.

Strict validation requires values to match the expected type more precisely.

Concept:

Normal Validation

"25"

↓
Convert
↓
25
↓
Accepted
Strict Validation

"25"
↓
Wrong exact type
↓
Rejected

Strict validation is useful when:

- Exact data types are important
- Silent conversion could create bugs
- Financial or critical systems need stronger guarantees
- Input contracts must be precise

Important Points

1. A Pydantic model defines the structure of valid data.
2. Pydantic models generally inherit from BaseModel.
3. Fields are defined using Python type annotations.
4. A field without a default value is required.
5. A field with a default value can be omitted.
6. A nullable field can contain None.
7. Nullable and omittable are different concepts.
8. Pydantic can convert compatible input values automatically.
9. Automatic type conversion is called type coercion.
10. Strict validation can be used when automatic conversion is undesirable.
11. A successfully created model contains validated and structured data.
12. Invalid input raises a validation error.

Syntax

Basic Model Structure

Import BaseModel

Create a class

Inherit from BaseModel

Define fields:

field_name : data_type

Create model instance

Pass input data

Pydantic validates the data

General pattern:

```
class ModelName(BaseModel):
```

```
    required_field: type
```

```
    default_field: type = default_value
```

```
    nullable_and_omittable_field: type | None = None
```

Validation process:

Input Data



Model Fields



Required Field Check



Type Validation



Type Conversion if Allowed



Default Value Assignment



Validated Model Object